

Google ADK Course Outline

1. Duration

40 Hours (5 days)

2. Objectives

By the end of this program, participants will be able to:

- Understand how Google ADK supports stateful, tool-using, and production-oriented AI agents.
- Design and build agents that combine tool use, retrieval, memory, and decision-making using Python.
- Implement RAG-based agents using ChromaDB and sentence-transformers with grounding and hallucination controls.
- Integrate MCP tools via FastMCP to connect agents to external systems.
- Use LiteLLM with OpenAI and OpenRouter to route across models with fallbacks and cost-aware routing.
- Add observability with LangSmith to trace agent runs, tool calls, RAG steps, and multi-agent delegation.
- Apply ReAct reasoning loops and reflection patterns to build structured decision-making agents.
- Implement input/output guardrails, prompt injection defences, and safe tool usage patterns.
- Instrument agents with structured logging, evaluation suites, and cost tracking.
- Build and ship a production-ready, fully-tested multi-agent eComBot system with CI/CD.

3. Audience

- Python Developers, QA Engineers, DevOps / Platform Engineers, AI/ML Engineers, and Solution Architects.
- Experience range: Freshers to 4–5 years, across QA, Dev, and DevOps roles.
- Suitable for teams moving from intermediate Python skills toward production-ready agent systems.

4. Pre-requisites

- Basic understanding of Python and running scripts or virtual environments.
- Familiarity with LLM concepts such as prompts and chat completion APIs is helpful but not mandatory.
- General awareness of HTTP APIs and JSON is useful.
- No JavaScript or frontend knowledge required — UI is built entirely in Python using Gradio.

5. Hardware & Network Requirements

Requirement	Minimum	Recommended
RAM	8 GB	16 GB
CPU	4-core (Intel i5 / Ryzen 5, 8th gen+)	8-core (i7 / Ryzen 7)
GPU	Not required	Optional (speeds up STT only)
Storage (free)	10 GB SSD	20 GB SSD
Internet	Stable broadband	Stable broadband
Microphone	Built-in	USB headset (better STT accuracy)

Note: All voice labs use CPU-optimised models and run comfortably on standard training laptops without a GPU.

6. Software Requirements

- Windows / macOS / Linux
- Latest Chrome or Edge browser
- VS Code with Python extension
- Python 3.11+
- Docker Desktop with Docker Compose
- Git
- OpenAI or OpenRouter account (API key required)
- LangSmith account (free tier API key required)

7. Tools & Platform

Tool	Type	Purpose
google-adk (Python)	Open Source	Core agent framework — agents, tools, orchestrators, workflows
ADK Web	Open Source	Built-in browser dev UI for building and debugging agents
OpenAI API (gpt-4o-mini / gpt-4o)	Requires API Key	Primary LLM backend for all agent interactions
OpenRouter	Requires API Key	Fallback and alternative model provider; access to multiple models via one key
LiteLLM	Open Source	Unified LLM gateway — model routing, fallbacks, retries, cost control
ChromaDB	Open Source	Vector database for RAG — product catalog and FAQ indexing
sentence-transformers	Open Source	Local embedding model — no API key required, runs on CPU
Redis	Open Source	Persistent session state store
FastMCP	Open Source	Build MCP-compliant tool servers in Python
Gradio	Open Source	Python-native chat UI with streaming and

		component rendering
faster-whisper	Open Source	Local real-time STT — CPU-optimised, multi-language
Piper TTS	Open Source	Lightweight local TTS — real-time on CPU
LiveKit Agents	Open Source	Python framework for STT → agent → TTS voice pipeline
LangSmith	Requires API Key	LLM observability — traces, cost, latency, eval runs
PromptFoo	Open Source	Agent evaluation framework — test suites, coverage, CI integration
Docker + Docker Compose	Open Source	Containerise and spin up the full local stack
GitHub Actions	Free Tier	CI/CD pipeline — lint, test, eval, build, deploy

8. Course Outline

Module 1: Introduction to ADK and Agentic AI

Scope:

- Agent concepts, lifecycle, and types
- ADK overview, architecture, and mental model
- Environment setup — API keys, .env, ADK Web
- Introduction to the eComBot storyline — electronics e-commerce support agent

Outcome: First working agent running locally via ADK Web.

Mandatory:

- Environment setup completed and verified
- First agent implemented and tested via ADK Web

Testing:

- Basic input/output validation
- Prompt behavior variations (minimum 3)

Project: eComBot v1 — Basic Support Agent

- Agent accepts a customer query as text input
- Returns a response based on intent — e.g. "Where is my order?" → "Let me check that for you."
- Domain: electronics e-commerce store (phones, TV decoders, accessories)

Module 2: LLMs in Agents — Tools, Memory, and State

Scope:

- Prompt engineering for agents
- Tool calling and function definitions
- Structured outputs
- Session state and memory — session_service, in-memory vs persistent (Redis)

Outcome: Task-solving agent with tool integration and stateful context.

Mandatory:

- At least 1 tool-integrated agent
- Structured response format demonstrated
- Basic session state demonstrated

Testing:

- Tool invocation correctness
- Failure scenarios (minimum 3 cases)
- State persistence across turns

Project: eComBot v2 — Tools + Memory

- Add tools: get_order_status(order_id), lookup_product(product_name)
- Agent returns structured output — product name, price, availability, description
- Session state: agent remembers customer name and last queried product across turns

Module 3: Retrieval Systems (RAG)

Scope:

- Embeddings and vector search using sentence-transformers and ChromaDB
- Context injection and chunking strategies
- Hallucination prevention

Outcome: Knowledge-grounded agent using RAG.

Mandatory:

- Agent responds only from grounded context
- Hallucination guard implemented

Testing:

- Minimum 15 queries (correct vs incorrect source match)
- Hallucination detection cases (minimum 3)

Project: eComBot v3 — Knowledge Base

- Build a product catalog and FAQ knowledge base
- Agent grounds all responses in the knowledge base — no hallucinated specs
- Unanswerable queries return a graceful fallback instead of fabricated answers

Module 4: LLM Gateway — Model Routing with LiteLLM

Scope:

- Model routing and provider abstraction using LiteLLM
- Cost optimisation strategies
- Fallback and retry logic with OpenAI and OpenRouter

Outcome: Multi-model routed agent with fallback behaviour.

Mandatory:

- Model switching and fallback implemented
- At least 2 providers configured (OpenAI + OpenRouter)

Testing:

- Model failure simulation
- Response consistency checks across providers

Project: eComBot v4 — Smart Model Routing

- Wrap the existing agent behind LiteLLM
- Route simple FAQ queries to a lighter/faster model (e.g. gpt-4o-mini)
- Route complex complaint or return-flow queries to a more capable model (e.g. gpt-4o)
- Fallback: if primary model fails, switch to OpenRouter automatically

Module 5: Tool Integration — MCP and FastMCP

Scope:

- External APIs, databases, and MCP tool servers using FastMCP
- Error handling and retries
- Auth patterns — API keys, basic security practices

Outcome: Agent with external integrations and graceful failure handling.

Mandatory:

- At least 1 live MCP tool integration via FastMCP
- Error handling for failures and invalid responses

Testing:

- API failure scenarios — timeouts, 500 errors
- Invalid / malformed response handling

Project: eComBot v5 — Live Integrations

- Integrate a mock order management MCP server: `get_order_status`, `cancel_order`, `get_invoice`
- Integrate a mock inventory MCP server: `check_stock(product_id)`
- All tools handle failures gracefully — timeouts, 404s, invalid IDs

Module 6: Multi-Agent Systems

Scope:

- Agent collaboration patterns
- Planner–Executor architecture
- Task delegation and orchestration

Outcome: Multi-agent system with minimum 3 agents and traceable communication.

Mandatory:

- Planner–Executor pattern implemented
- Task delegation across agents demonstrated

Testing:

- Agent communication validation
- Trace-based evaluation of delegation flow

Project: eComBot v6 — Multi-Agent Architecture

- Split the single agent into a coordinated system; the original agent becomes the Orchestrator
- Orchestrator Agent — classifies incoming intent, delegates to the right sub-agent
- Support Agent — handles order queries, complaints, returns (carries all M1–M5 work)
- Sales Agent — handles product discovery, recommendations, comparisons, upsells
- Routing: "Where is my order?" → Support Agent / "What phone should I buy?" → Sales Agent

Module 7: Generative UI

Scope:

- What Generative UI means in the context of AI agents
- Rendering structured agent responses as interactive UI components using Gradio
- Streaming UI updates in real time as the agent responds
- Connecting agent output schema to UI components

Outcome: A Python chat interface that renders rich, dynamic UI components driven by agent responses.

Mandatory:

- At least 3 distinct UI components rendered from agent output
- Real-time streaming of agent response into UI
- UI reflects agent routing (which agent responded)

Testing:

- Component renders correctly for each agent response type
- Graceful fallback when agent returns unstructured output
- Streaming behaviour validated (no full-page re-renders)

Project: eComBot v7 — Chat UI

- Chat interface: input box, message history, agent name display
- Product card — name, price, stock badge, image placeholder (triggered by Sales Agent)
- Order status card — order ID, status pill, estimated delivery, cancel button (triggered by Support Agent)
- Agent routing trace — shows which agent handled the request and why
- Source tag — on RAG responses, shows which knowledge base document was used
- Model badge — subtle indicator of which model answered (from LiteLLM routing)

Module 8: Voice AI — Real-Time

Scope:

- Real-time STT using faster-whisper (small model, CPU int8)
- Real-time TTS using Piper TTS
- Streaming audio pipeline: mic → STT → agent → TTS → speaker via LiveKit Agents
- Latency considerations and interruption handling

Outcome: Voice-enabled eComBot with real-time interaction.

Mandatory:

- Real-time STT → agent → TTS pipeline implemented end-to-end
- At least 2 languages demonstrated (e.g. English + Hindi or French)
- Latency measured and within acceptable range

Testing:

- Latency checks (STT + agent + TTS round trip)
- Speech accuracy validation across both languages
- Interruption / mid-sentence handling

Project: eComBot v8 — Voice Interface

- Customer speaks a query — STT transcribes it in real time
- Orchestrator processes and responds — TTS reads it back
- Same multi-agent routing works over voice
- UI: mic button on chat interface, live transcription shown as customer speaks

Module 9: Advanced Design Patterns — ReAct and Reasoning Loops

Scope:

- ReAct, Reflection, and reasoning loops
- When and why to use each pattern
- Structured step-by-step reasoning before responding

Outcome: Agent with at least one structured reasoning loop.

Mandatory:

- One reasoning loop implementation (ReAct or Reflection)

Testing:

- Decision correctness
- Loop termination behaviour

Project: eComBot v9 — Reasoning Sales Agent

- Upgrade the Sales Agent with a ReAct reasoning loop
- Sales Agent reasons step by step: identify budget → filter catalog → compare specs → recommend
- Reflection: if a recommendation is rejected, agent reflects and adjusts
- UI: collapsible "Agent Reasoning" panel showing thought steps before the final answer

Module 10: MLOps for Agents — Observability and Evaluation**Scope:**

- Structured logging and observability using LangSmith
- Prompt and version control practices
- Latency, cost, and performance monitoring
- Agent evaluation with PromptFoo

Outcome: Deployable, observable agent service.

Mandatory:

- LangSmith logging and monitoring setup
- At least one evaluation run via PromptFoo

Testing:

- Latency and cost profiling
- Evaluation coverage across key flows

Project: eComBot v10 — Observability Layer

- Structured logs on every agent call: intent, agent selected, model used, latency, cost
- PromptFoo eval suite: minimum 10 test cases covering support flows, sales flows, and edge cases
- Cost tracking: spend per model per session visible in LangSmith
- UI: hidden admin panel (toggle) showing live logs, eval results, and cost per session

Module 11: Security and Guardrails

Scope:

- Prompt injection attack patterns
- Output filtering and content policies
- Safe tool usage and boundary enforcement

Outcome: Secure, governed agent.

Mandatory:

- At least one guardrail implementation covering both input and output

Testing:

- Attack simulations (minimum 3 injection attempts)
- Policy enforcement validation

Project: eComBot v11 — Hardened Agent

- Input guardrail: detect and block prompt injection attempts
- Output guardrail: filter for PII leakage, off-topic content, and competitor mentions
- Tool safety: validate all tool inputs before execution — no raw user input passed directly to APIs
- UI: "Blocked" indicator in chat when a guardrail fires, with reason shown

Capstone: End-to-End eComBot System

Objective: The agent built incrementally across all 11 modules is the capstone. Wire it together, write the pipeline, and ship it.

What Participants Have by Module 11:

- Multi-agent architecture: Orchestrator + Support + Sales agents
- RAG over product catalog and FAQ
- Live API/tool integrations via FastMCP (orders, inventory)
- LiteLLM model routing with OpenAI primary and OpenRouter fallback
- Generative UI with rich components (Gradio)
- Real-time voice interface (STT → agent → TTS)
- ReAct reasoning loop in the Sales Agent
- Structured logging, eval suite, and cost tracking via LangSmith + PromptFoo
- Input/output guardrails and tool safety

Capstone Deliverables:

Area	Requirement
Agent System	Orchestrator + Support + Sales agents wired together
Knowledge	RAG over product catalog and FAQ
Tools	Order API + Inventory API with full error handling via FastMCP
Voice	Real-time voice interface (minimum 1 language)
UI	Generative chat UI — product cards, order cards, routing trace, reasoning panel
Observability	LangSmith logs + PromptFoo eval suite (minimum 10 cases)
Security	Input and output guardrails active
CI/CD	GitHub Actions pipeline: lint → test → eval → build
Deployment	Optional — containerised Docker image or cloud deploy

Evaluation Criteria:

- Functional correctness — does the agent handle support and sales flows end-to-end?
- Traceability — can you follow the routing, reasoning, and tool calls in LangSmith?
- Test coverage — are critical paths validated in the PromptFoo eval suite?
- Pipeline — does the CI/CD pipeline run clean end-to-end?
- Production readiness — guardrails active, errors handled, cost tracked

9. eComBot — Progressive Build Summary

One agent, built iteratively across every module. Each module adds a layer; nothing is discarded.

Module	What Gets Added	Agent State
M1	Basic chat agent, intent-based responses	eComBot v1 — single agent
M2	Tools + structured output + session memory	eComBot v2 — stateful, tool-calling
M3	RAG over product catalog and FAQ	eComBot v3 — knowledge-grounded
M4	LiteLLM model routing and fallback	eComBot v4 — multi-model
M5	FastMCP order and inventory integrations	eComBot v5 — externally connected
M6	Split into Orchestrator + Support + Sales agents	eComBot v6 — multi-agent
M7	Generative UI — chat interface and rich components	eComBot v7 — UI layer
M8	Real-time voice interface (STT → agent → TTS)	eComBot v8 — voice-enabled
M9	ReAct reasoning loop in Sales Agent	eComBot v9 — reasoning
M10	Logging, cost tracking, PromptFoo evals	eComBot v10 — observable
M11	Input/output guardrails, tool safety	eComBot v11 — production-hardened
Capstone	CI/CD pipeline and optional deployment	eComBot Final — shipped